

PROGETTO SDCC – TRACCIA B4
APPLICAZIONE A MICROSERVIZI

Federico Pepe – 0365363

Anno Accademico 2025 - 2026

Roadmap



ARCHITETTURA SISTEMA

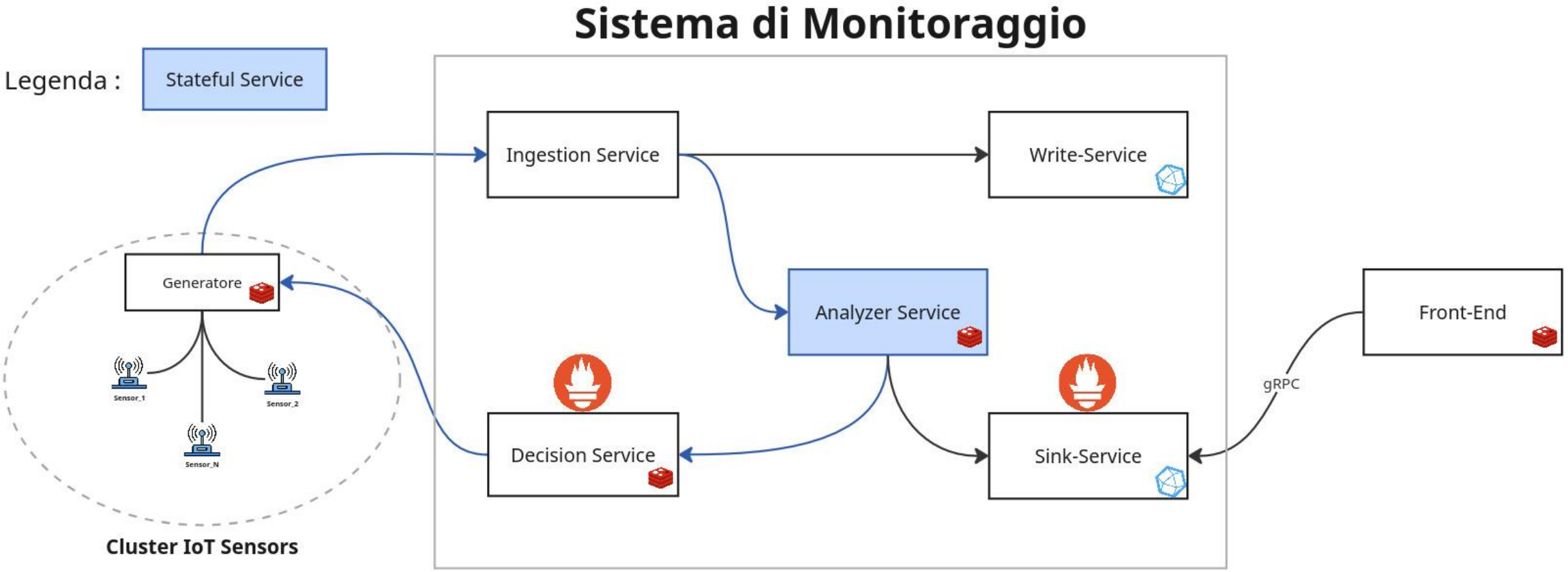


DEPLOYMENT



ANALISI PRESTAZIONALE

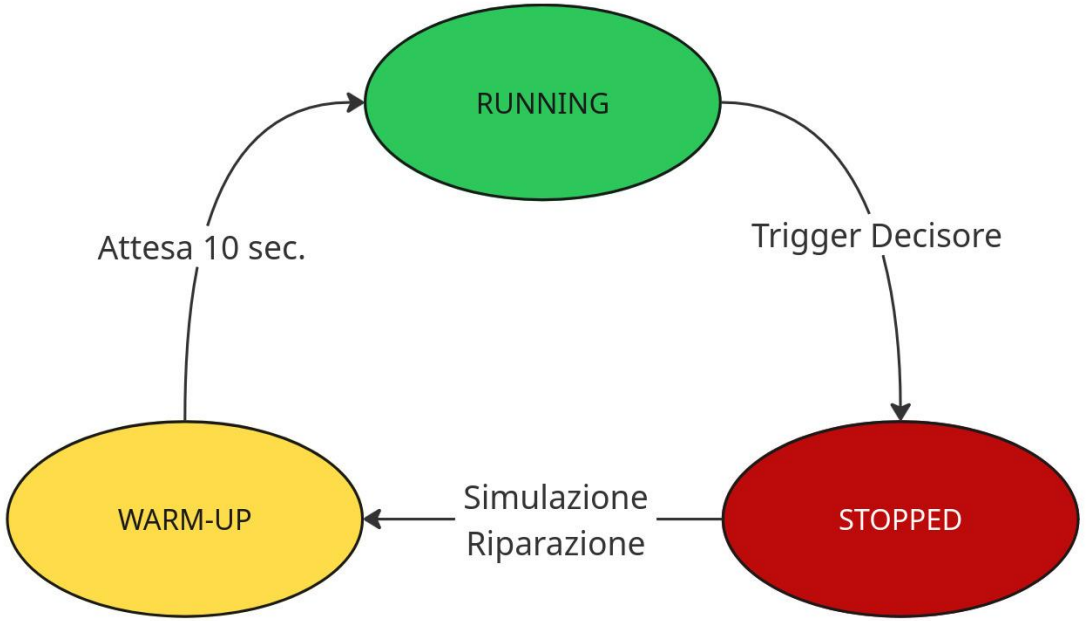
Architettura Sistema



Architettura Sistema - Generatore



- **Generazione dei dati**
  - Definizione di due specifiche tipologie di sensore:
    - *TemperatureSensor, PressureSensor.*
  - Per ciascun sensore, si definiscono i parametri operativi.
  - Definizione di tre comportamenti operativi differenti.
- **Gestione dello stato dei sensori.**
- **Scrittura sul Topic**
  - Pubblicazione sui topic Kafka:
    - *temperature-topic-sensor,*
    - *pressure-topic-sensor*



Architettura Sistema – Microservizi

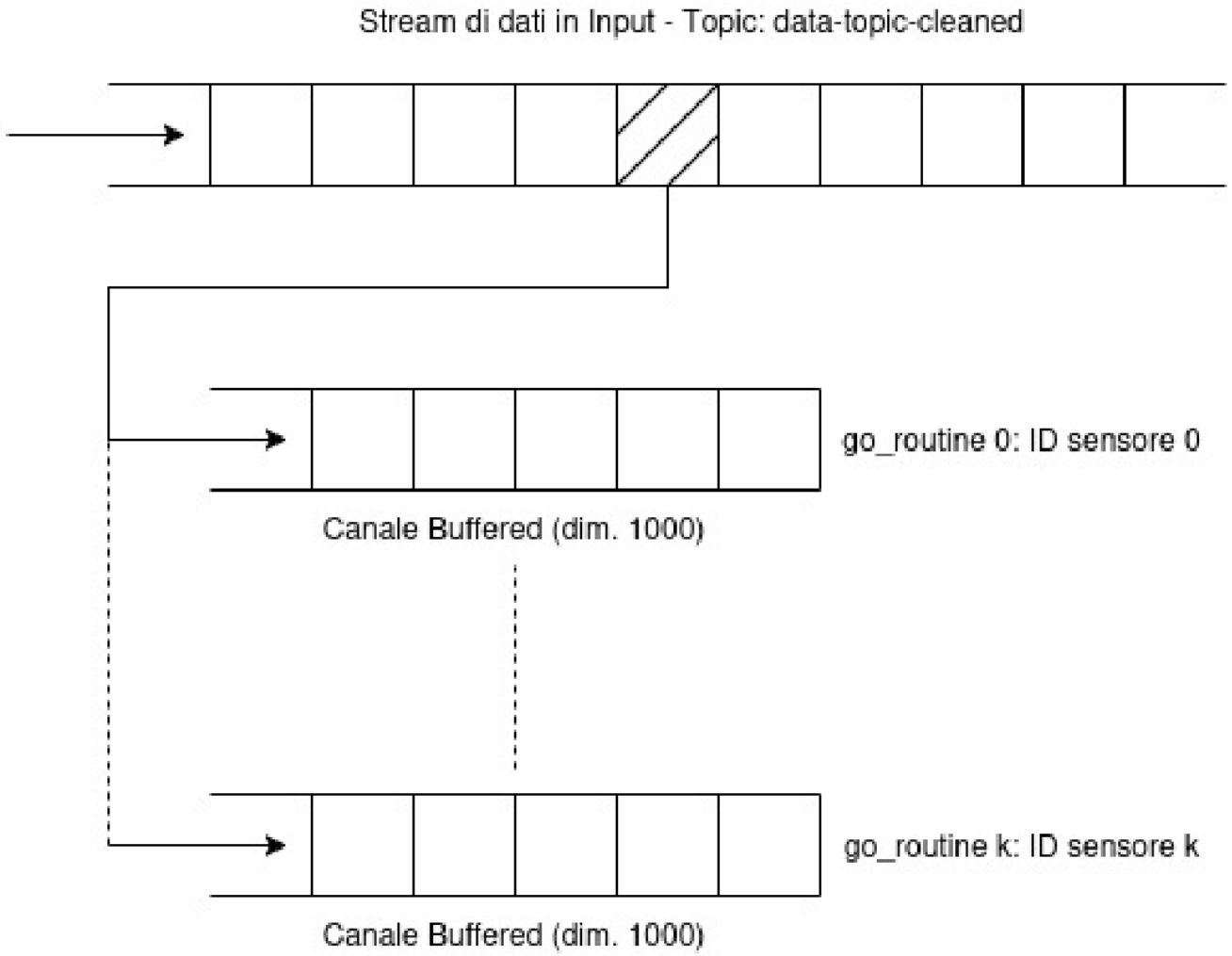
- **Ingestion-Service**
  - Pulizia dei Dati
    - Esecuzione del data cleaning
    - Normalizzazione delle strutture dati
  - Risultati pubblicati sul topic *data-topic-cleaned*.
- **Decision-Service**
  - Modulo di monitoraggio e reattività, in tempo reale.
  - Per ogni evento, si valuta una potenziale anomalia verificando:
    - Soglie operative.
    - Instabilità.
    - Deriva Temporale, ovvero il tasso di variazione.
  - In caso di violazione, si invia un messaggio di **allarme** al generatore.

Architettura Sistema – Microservizi

- **Analyzer-Service**
  - Responsabile dell'elaborazione analitica dei dati sensoriali.
  - Strutturato su due livelli principali:
    - *Dispatcher* centrale.
    - Una serie di Worker concorrenti, allocati dinamicamente.

Architettura Sistema – Microservizi

- Dispatcher centrale



Architettura Sistema – Microservizi

- **Dispatcher centrale**
  - Gestione della concorrenza:
    - Definizione di una Mappa (sync.Map), per tracciare sensori correntemente attivi
    - Al rilevamento di un nuovo sensore:
      - Inserimento nella Mappa, ed avvio di una nuovo go-routine.
      - Associazione di un canale bufferizzato (InputChannel).
  - Tolleranza ai guasti:
    - Definizione di un checkpointing periodico (ogni 30 sec.) in due fasi:
      - Acquisizione di uno Snapshot, per ogni sensorWorker attivo.
      - Salvataggio dello stato su Redis.
      - In caso positivo, esecuzione del commit degli *offset,* per singole partizioni.
  - Gestione degli eventi di controllo:
    - Sottoscrizione al canale Pub/Sub Redis
    - In presenza di eventi di cambio di stato:
      - Notifica il sensorWorker, tramite un canale dedicato, sospendendo l'elaborazione.

Architettura Sistema – Microservizi

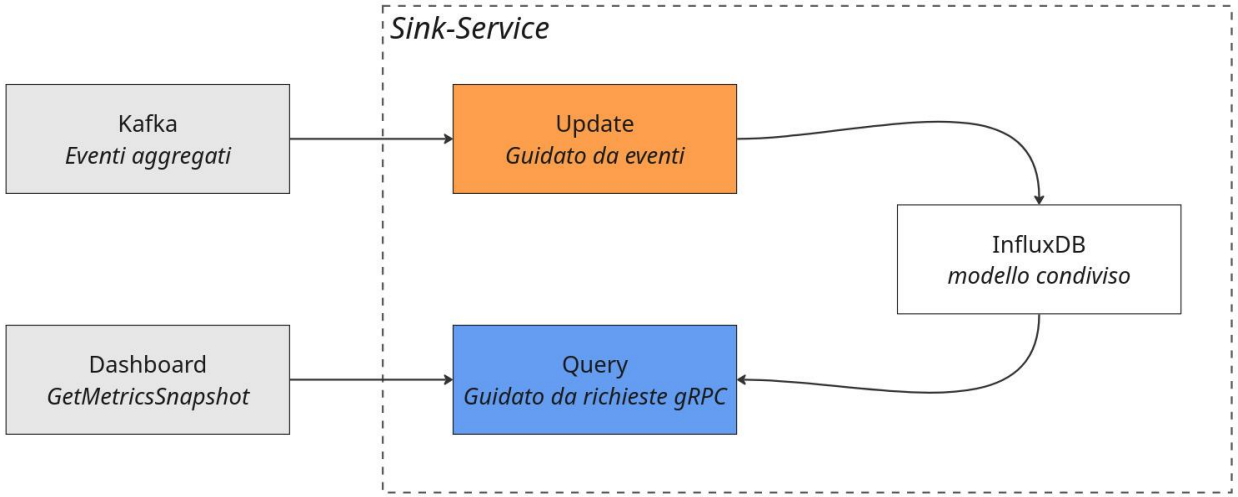
- **WorkerInstance**
  - **Definizione della finestra:**
    - Sliding-Window:
      - *Periodo di osservazione: 60 sec.*
      - *Passo di Slide: 30 sec.*
  - **Definizione del Watermark**
    - Definita una strategia di tipo Bounded-Out-Of-Ordeness.
  - **Processamento della finestra:**
    - Algoritmo di Welford (media, dev. Std.)
    - Regressione Lineare (tasso di variazione)
    - Pubblicazione sul Topic Kafka *processed-data-topic*
  - **Acquisizione dello stato (Checkpointing)**
  - **Gestione Transizioni di Stato:**
    - Worker in ascolto sul canale *invalidDataChannel*
    - Se stato → **STOPPED**
      - Salvataggio del timestamp (stopTime) su Redis
      - Impostiamo flag isStopped = true
      - Scarta tutti gli eventi con timestamp t > stopTime
    - Quando lo stato da **STOPPED** → **WARM-UP**
      - Resetta a zero tutti i contatori temporali
      - Azzeramento del buffer della finestra.

Architettura Sistema – Write & Sink

Write Service	Sink Service
Consuma da data-topic-cleaned	Consuma da processed-data-topic
Persiste letture grezze (storico puntuale)	Persiste dati aggregati di finestra
Scrittura asincrona su InfluxDB	Aggiorna il proprio *real-model* su InfluxDB + Espone gRPC per la Dashboard

- Non sono due viste dello stesso dato, sono due livelli d'astrazione distinti.

Architettura Sistema – Pattern CQRS



- **Percorso Command**: Consuma processed-data-topic e scrive su InfluxDB.
- **Percorso Query**: La chiamata gRPC legge dallo stesso data store.
- Carico asimmetrico, con scritture continue e letture "sporadiche" dalla Dashboard.
- Lettura semplificata, poiché i dati sono già stati aggregati.
- Disaccoppiamento dei due flussi.

Architettura Sistema – Apache Kafka

- **Dorsale di messaggistica asincrona per i dati di telemetria**
  - Trasferimento della logica di calcolo sui Microservizi, demandando a Kafka la gestione dello stato e dell'instradamento dei messaggi.
  - Tolleranza ai guasti:
    - Gestione della lettura dei dati, tramite il meccanismo Fetch-Commit
    - Garanzie di riproducibilità dei dati.
- **Per ogni fase è stato definito un topic specifico.**
  - *Temperature/pressure-topic-sensor*, con 6 partizioni
  - *Data-topic-cleaned, processed-data-topic,* con 6 partizioni
  - *Signals-topic*, con 1 partizione

Architettura Sistema – Redis & InfluxDB

- **Redis:**
  - Datastore in-memory, per la consultazione dei parametri operativi dei sensori.
  - Sfruttato Redis Pub/Sub come motore di notifica, verso UI e Generatore
    - Notifiche effimere
      - Aggiornamenti alla Dashboard sono dati temporanei
      - Eliminazione della gestione degli Offset e Consumer-Group
- **InfluxDB:**
  - Database di tipo TSDB, perfetto per la gestione nativa di dati telemetrici temporizzati.
  - Sfruttato in due casi principali:
    - Storicizzazione ad alto throughput (Write-Service)
    - Persistenza e consultazione (Sink-Service)

Roadmap



ARCHITETTURA SISTEMA



DEPLOYMENT



ANALISI PRESTAZIONALE

Deployment – Kubernetes k3s

- Si è scelto una distribuzione lightweight di Kubernetes, per minimizzare il footprint di risorse sui nodi, massimizzando la memoria e CPU allocabile.
- **Topologia del Cluster e Separazione dei ruoli (5 Istanze):**
  - Control Plane (Master):
    - Dedicato all'API Server, allo scheduler ed al control manager.
  - Worker 1 (load):
    - Dedicato all'events-generator.
  - Worker 2 (core):
    - Riservato all'elaborazione CPU-bound dei servizi (Ingestion, Analyzer, Decision)
  - Worker 3 (data):
    - Riservato ad InfluxDB e servizi di scrittura (sink, write)
  - Worker 4 (Kafka):
    - Ospita Apache Kafka ed il relativo exporter

Deployment – Horizontal Pod Autoscaler

- HPA è un controller di Kubernetes che adatta automaticamente il n° di repliche (POD), in base al carico di lavoro rilevato. Ogni 15 sec. (defualt) esegue:

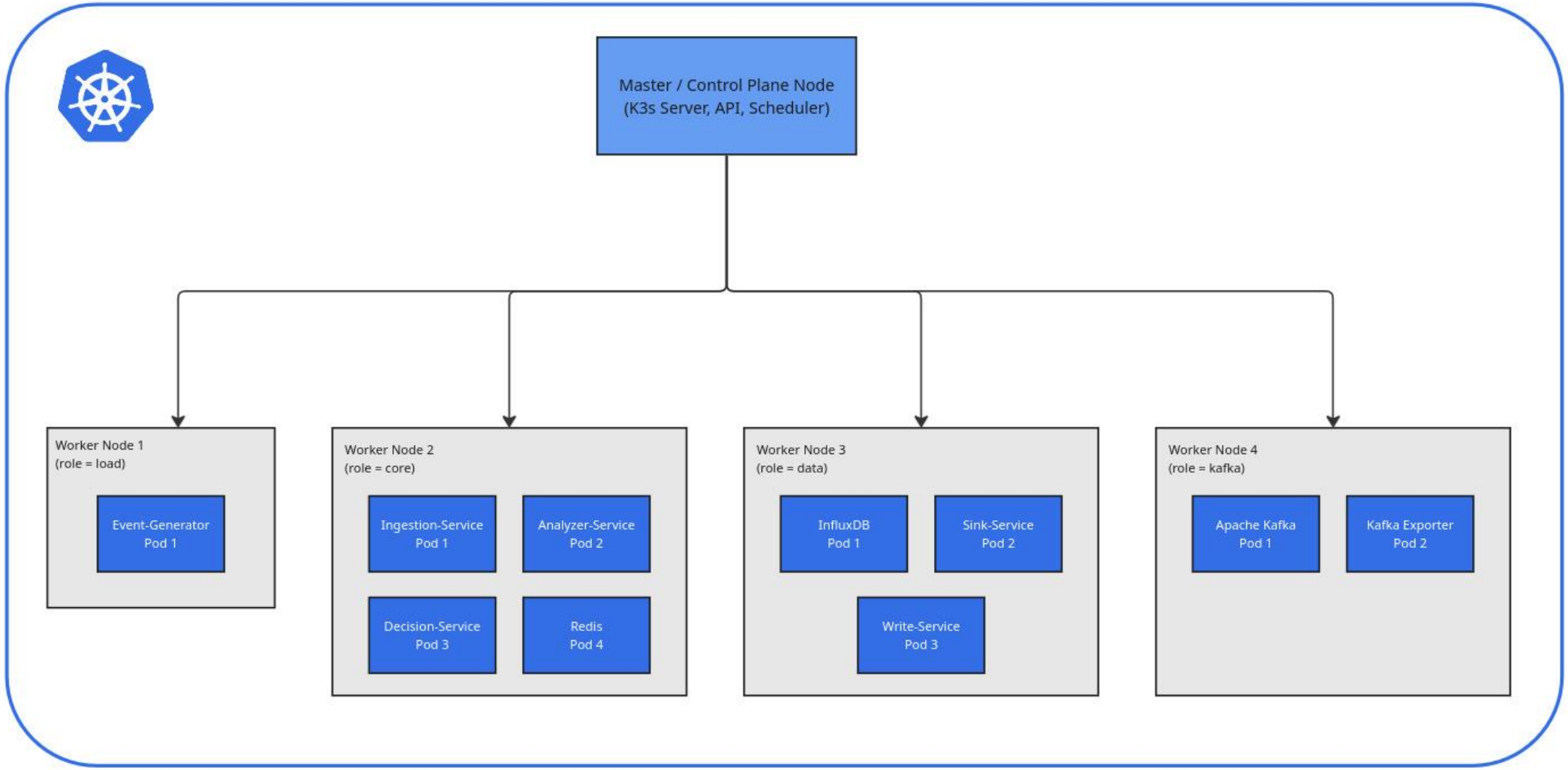
$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \cdot \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \right\rceil$$

- Inoltre consente di configurare il comportamento dello scaling, attraverso il blocco *behavior, definendo:*
  - *StabilizationWindowSeconds*
    - Periodo di tempo in cui HPA osserva lo storico delle metriche.
  - *Policies*
    - Regole che definiscono **quanto** velocemente aggiungere/rimuovere un Pod in un dato intervallo di tempo.
  - *SelectPolicy*
    - Determina quali policy applicare se sono definite più regole contemporaneamente.

Deployment – Horizontal Pod Autoscaler

- **Definizione dei parametri**
  - Utilizza sia metriche di risorsa che metriche custom applicative.
  - Soglie e metriche Target*:*
    - CPU: 70%
    - Memoria: 80%
    - Kafka Consumer Lag: Valutazione sull'accodamento nei topic
      - Analyzer: AverageValue per Pod pari a 5K messaggi
      - Decision ed Ingestion: AverageValue per Pod pari a 1K messaggi
  - Range di scalabilità: Da 1 fino ad un massimo di 6 Pod, per servizio.
  - Policy di stabilizzazione:
    - Scale-Up: Espansione immediata che raddoppia le repliche, al superamento delle soglie
    - Scalde-Down: Periodo d'osservazione di 3 minuti, con contrazione graduale del -25% (sulle repliche attuali) ogni 60 sec.

Deployment – Kubernetes k3s



Roadmap



ARCHITETTURA SISTEMA



DEPLOYMENT



ANALISI PRESTAZIONALE

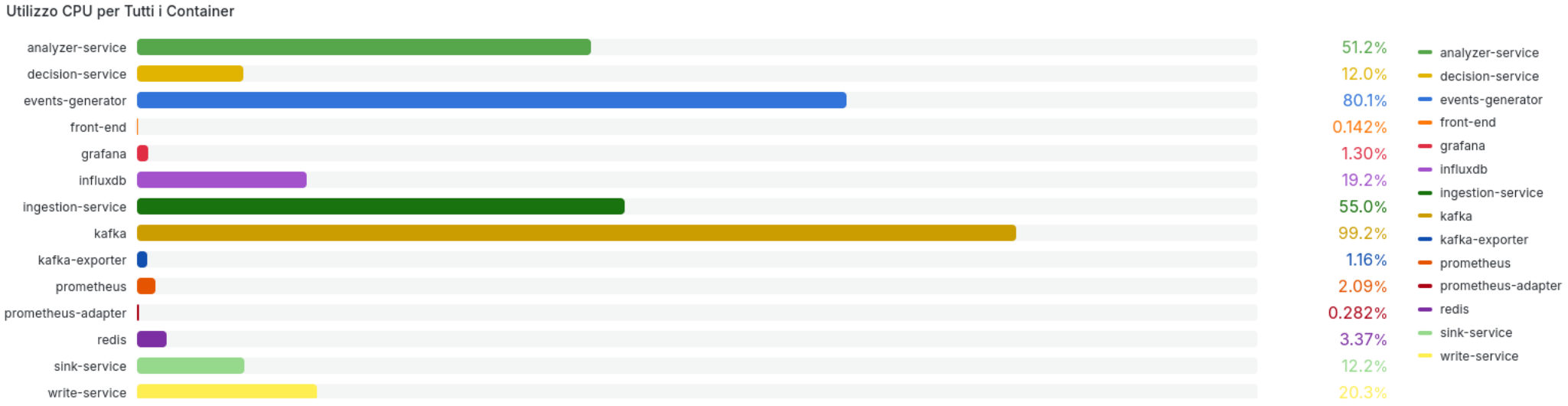
Analisi Prestazionali - Metodologia

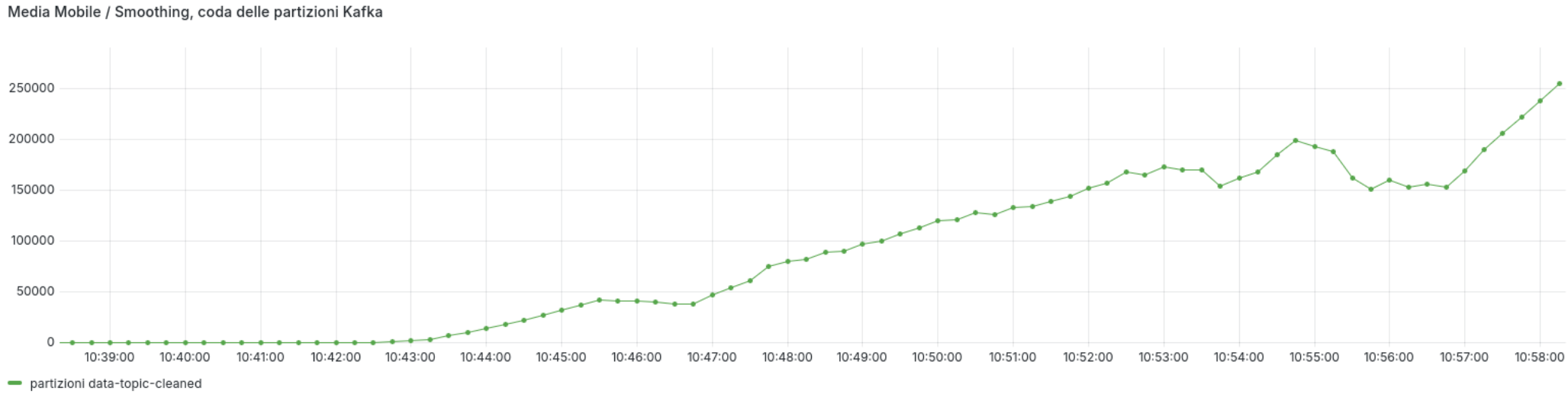
- Valutazione sperimentale dei tempi **Latenza** *end-to-end,* **Throughput** e **Backpressure** dell'applicazione di *Stream-Processing*.
- **Raccolta Metriche:**
  - Sfruttato il framework **Prometheus**, con scraping ogni 5 sec.
  - **Throughput :**
    - Monitoraggio del contatore vettoriale: *ProcessedEventsTotal*
    - Sfruttato finestre mobili di 1 minuto, per il calcolo del rate
  - **Latenza :**
    - **End-to-end :** dall'events-generator fino ai rispettivi sink di destinazione (Decision e Sink Service).
    - Sfruttato *Latency-Marker* ogni 0.5 secondi.
    - Media aggregata su una finestra di 1 minuto.
  - **Backpressure:**
    - Misuriamo il tempo di stazionamento del messaggio in coda, prodotto dall'Analyzer-Service.

Analisi Prestazionali - Configurazione

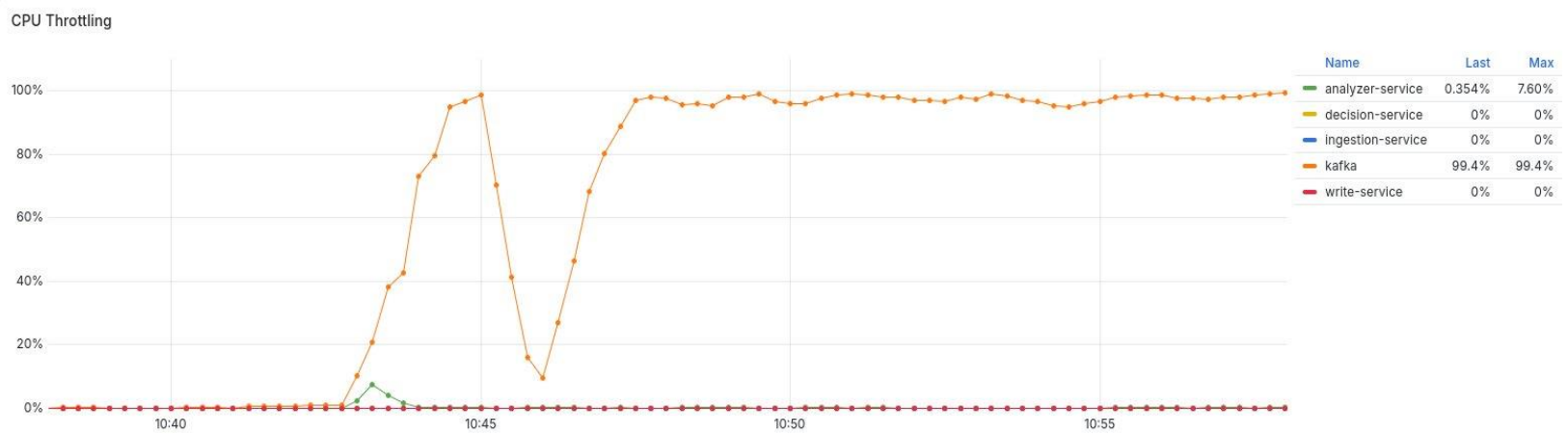
- L'infrastruttura del Cluster k3s è distribuita su istanze EC2 di tipo *t3medium.*
- Al fine di individuare il limite operativo del sistema si è scelto di configurare Apache Kafka su singolo Broker, permettendo di:
  - Isolare la capacità massima di smaltimento del bus di messaggistica
  - Valutare l'efficacia delle politiche di scaling orizzontale (HPA)
  - Monitorare la stabilità dell'architettura al raggiungimento della saturazione delle risorse del sistema.

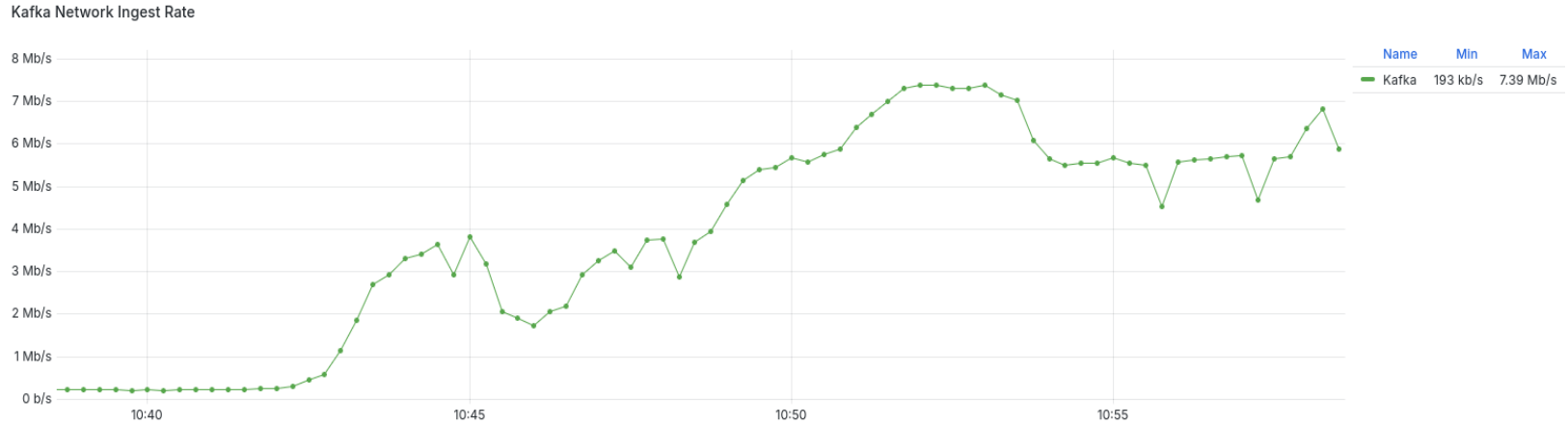
Analisi Prestazionali – CPU & Accodamento





Analisi Prestazionali – CPU Broker Kafka





99.2%
Saturazione CPU Broker

7.39 Mb/s
Traffico di rete
Contenuto



Il collo di bottiglia è puramente di natura computazionale, non di I/O di rete

Analisi Prestazionali – HPA e Reattività del carico

analyzer-service-hpa	Deployment/analyzer-service	cpu: 48%/70%, memory: 3%/80% + 1 more...	1	6	1	19m
decision-service-hpa	Deployment/decision-service	cpu: 0%/70%, memory: 4%/80% + 1 more...	1	6	3	19m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 3%/70%, memory: 24%/80%	1	6	1	19m
decision-service-hpa	Deployment/decision-service	cpu: 0%/70%, memory: 5%/80% + 1 more...	1	6	2	20m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 4%/70%, memory: 25%/80%	1	6	1	20m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 54%/70%, memory: 2%/80% + 1 more...	1	6	1	20m
decision-service-hpa	Deployment/decision-service	cpu: 1%/70%, memory: 8%/80% + 1 more...	1	6	1	21m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 11%/70%, memory: 25%/80%	1	6	1	21m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 49%/70%, memory: 6%/80% + 1 more...	1	6	1	21m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 25%/70%, memory: 33%/80% + 1 more...	1	6	1	22m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 184%/70%, memory: 25%/80%	1	6	1	22m
decision-service-hpa	Deployment/decision-service	cpu: 11%/70%, memory: 9%/80% + 1 more...	1	6	1	22m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 66%/70%, memory: 47%/80% + 1 more...	1	6	2	22m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 209%/70%, memory: 24%/80%	1	6	2	22m
decision-service-hpa	Deployment/decision-service	cpu: 11%/70%, memory: 6%/80% + 1 more...	1	6	2	22m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 89%/70%, memory: 53%/80% + 1 more...	1	6	3	22m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 124%/70%, memory: 24%/80%	1	6	3	23m
decision-service-hpa	Deployment/decision-service	cpu: 5%/70%, memory: 6%/80% + 1 more...	1	6	2	23m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 77%/70%, memory: 41%/80% + 1 more...	1	6	6	23m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 93%/70%, memory: 24%/80%	1	6	3	23m
decision-service-hpa	Deployment/decision-service	cpu: 13%/70%, memory: 7%/80% + 1 more...	1	6	2	23m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 71%/70%, memory: 49%/80% + 1 more...	1	6	6	23m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 107%/70%, memory: 24%/80%	1	6	4	23m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 50%/70%, memory: 54%/80% + 1 more...	1	6	6	23m
decision-service-hpa	Deployment/decision-service	cpu: 14%/70%, memory: 7%/80% + 1 more...	1	6	2	23m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 105%/70%, memory: 25%/80%	1	6	5	23m
decision-service-hpa	Deployment/decision-service	cpu: 17%/70%, memory: 6%/80% + 1 more...	1	6	2	23m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 49%/70%, memory: 59%/80% + 1 more...	1	6	6	23m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 105%/70%, memory: 25%/80%	1	6	5	24m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 53%/70%, memory: 65%/80% + 1 more...	1	6	6	24m
decision-service-hpa	Deployment/decision-service	cpu: 5%/70%, memory: 7%/80% + 1 more...	1	6	2	24m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 113%/70%, memory: 25%/80%	1	6	5	24m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 61%/70%, memory: 75%/80% + 1 more...	1	6	6	24m
decision-service-hpa	Deployment/decision-service	cpu: 13%/70%, memory: 7%/80% + 1 more...	1	6	4	24m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 111%/70%, memory: 25%/80%	1	6	5	24m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 64%/70%, memory: 66%/80% + 1 more...	1	6	6	24m
decision-service-hpa	Deployment/decision-service	cpu: 10%/70%, memory: 7%/80% + 1 more...	1	6	4	24m
decision-service-hpa	Deployment/decision-service	cpu: 5%/70%, memory: 11%/80% + 1 more...	1	6	6	36m
analyzer-service-hpa	Deployment/analyzer-service	cpu: 105%/70%, memory: 148%/80% + 1 more...	1	6	6	36m
ingestion-service-hpa	Deployment/ingestion-service	cpu: 91%/70%, memory: 25%/80%	1	6	5	36m

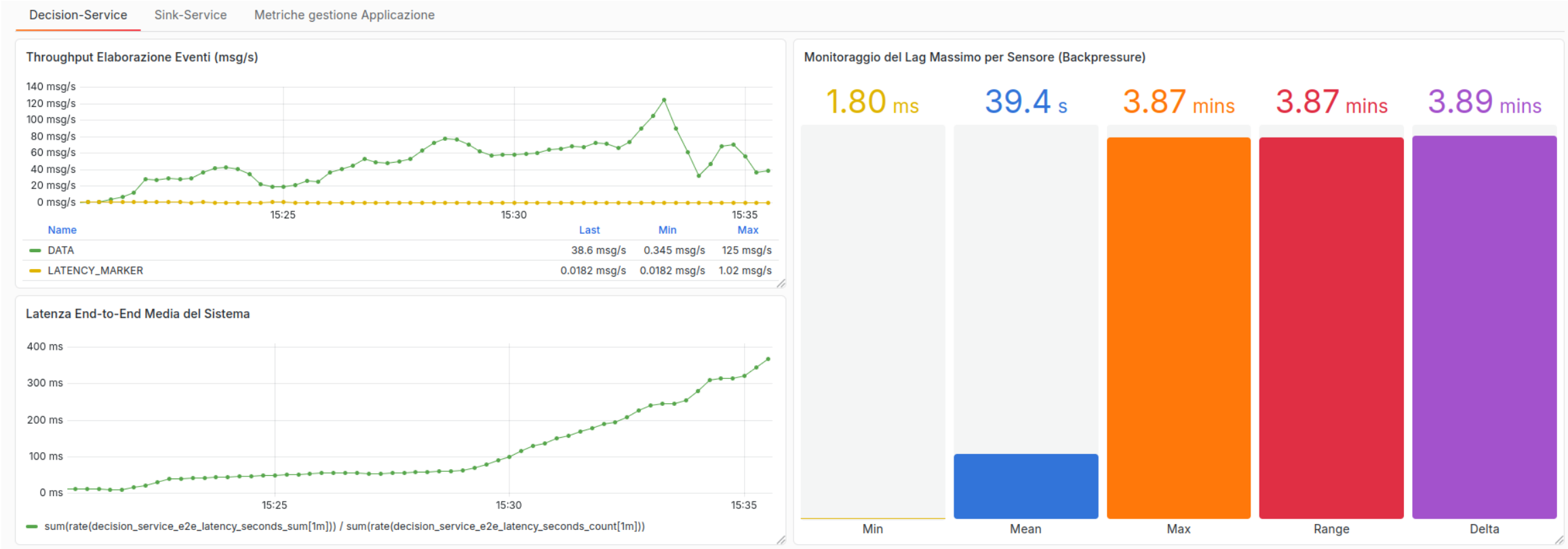
Repliche Massime

- Servizi scalati a 6 repliche. Ingestion-Service fermo a 5

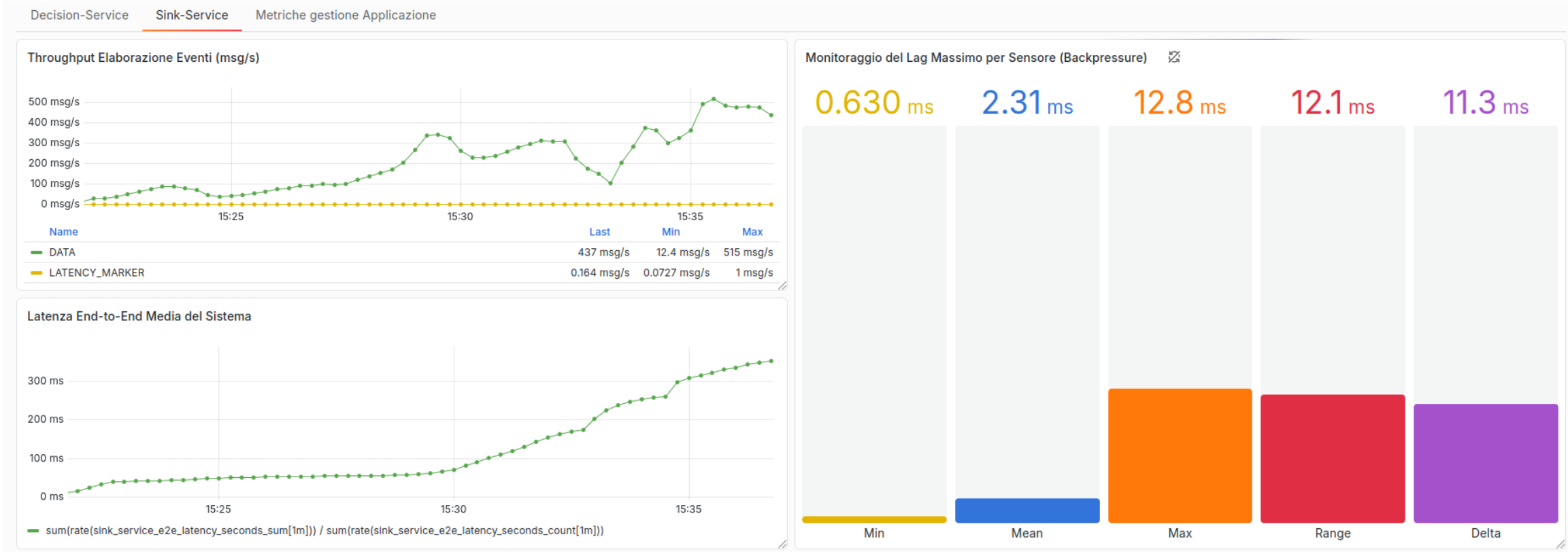
Picchi di CPU

- Ingestion-Service al 209% rispetto al target HPA del 70%

Analisi Prestazionali – Decisore



Analisi Prestazionali – Sink



Conclusioni

- **Corretta attivazione dell'HPA**
  - Tutti e tre i servizi hanno reagito all'aumento progressivo del workload.
  - Ogni evento di Scale-Out ha causato cali temporanei di throughput, dovuto al *rebalance* dei consumer group di Kafka.
- **Collo di Bottiglia sul Broker Kafka, di natura Computazione**
  - Esclusivamente a livello CPU, arrivando a completa saturazione.
- **Impatto della saturazione diversificato**
  - L'asimmetria tra Decision e Sink service, riflette il diverso costo computazionale per messaggio.
  - La saturazione del Broker agisce da moltiplicatore, penalizzando il consumer con l'elaborazione più onerosa.



Grazie per l'Attenzione !

Federico Pepe